

Architecture

- Une *base de données* constitue un ensemble structuré d'informations. Son interrogation s'effectue généralement via un *SGBD* (Système de Gestion de Bases de Données) exploitant le langage standardisé SQL.
- L'architecture matérielle se décline selon le volume et le partage des données :
 - *Architecture à 1 niveau* : fonctionnement local, sans partage.
 - *Architecture à 2 niveaux* (client-serveur) : le serveur héberge les données, le client envoie les requêtes et peut gérer une partie du traitement.
 - *Architecture à 3 niveaux* (ou 3 tiers) : sépare le client (affichage), le serveur d'application (traitement logique et requêtes) et le serveur de base de données (stockage).

Vocabulaire et modélisation

- Les données sont organisées sous forme de *tables* (ou *relations*).
- Une ligne de la table est appelée un *enregistrement* (ou *tuple*, ou *vecteur*). Elle représente un objet unique.
- Une colonne est appelée un *attribut*.
- L'ensemble des valeurs autorisées pour un attribut constitue son *domaine*. L'ensemble des attributs définit le *schéma* de la table.
- L'identification unique d'un enregistrement est garantie par la *clé primaire* (souvent un identifiant abstrait de type entier, noté *id*).
- La *clé étrangère* est un attribut faisant référence à la clé primaire d'une autre table, permettant de lier les données entre elles.
- La modélisation des liens (*cardinalités*) détermine la structure :
 - *Relation 1 à 1* : correspondance bijective (rare).
 - *Relation 1 à plusieurs* : modélisée par l'ajout d'une clé étrangère dans la table portant la multiplicité.
 - *Relation plusieurs à plusieurs* : impose la création d'une *table d'association* (ou de liaison) contenant au minimum deux clés étrangères référençant les tables concernées.

Extraction et restriction

- Toute instruction SQL s'achève par un point-virgule.
- La *projection* consiste à sélectionner des colonnes spécifiques au moyen de la clause **SELECT**. Le caractère ***** permet de sélectionner l'intégralité des attributs.
- La notation pointée **table.attribut** est requise pour lever toute ambiguïté lors de la manipulation de multiples tables.
- La *restriction* filtre les lignes vérifiant une condition booléenne via la clause **WHERE**.
- Les opérateurs de comparaison classiques sont **=**, **<>** (différent de), **>**, **<**, **>=**, **<=**. Ils se combinent avec les opérateurs logiques **AND**, **OR**, **NOT**.

```
1 SELECT titre, artiste
2 FROM chanson
3 WHERE duree > 180 AND artiste = 'Agar Agar';
```

- L'opérateur `LIKE` permet la recherche de motifs dans les chaînes de caractères. Le caractère `_` se substitue à un unique caractère exact, tandis que `%` (placé au début, au milieu ou à la fin) remplace une séquence de zéro, un ou plusieurs caractères.

```
1 SELECT artiste FROM chanson WHERE nom LIKE '%life%';
```

Tris, limites et opérations ensemblistes

- Le tri des résultats s'opère avec `ORDER BY`, suivi de `ASC` (croissant, par défaut) ou `DESC` (décroissant). L'ordre s'applique également de manière lexicographique sur les chaînes de caractères.
- L'élimination des doublons dans le résultat d'une projection s'effectue en plaçant le mot-clé `DISTINCT` immédiatement après `SELECT`.
- La limitation du nombre de résultats renvoyés utilise `LIMIT`. Le décalage de l'indice de départ s'obtient avec `OFFSET` (l'indice initial étant 0).

```
1 SELECT DISTINCT artiste
2 FROM chanson
3 ORDER BY artiste ASC
4 LIMIT 4 OFFSET 1; -- du deuxième au cinquième
```

- Les opérations ensemblistes exigent que les tables manipulées aient des schémas compatibles (même nombre d'attributs et types compatibles). Elles incluent :
 - `UNION` : fusionne les lignes des deux tables.
 - `INTERSECT` : conserve les lignes communes.
 - `EXCEPT` : conserve les lignes de la première table absentes de la seconde.

```
1 SELECT * FROM playlist_A
2 UNION
3 SELECT * FROM playlist_B;
```

Croisement de données et jointures

- Le *produit cartésien* génère toutes les combinaisons possibles entre les lignes de plusieurs tables. Il s'obtient en listant les tables séparées par des virgules dans la clause `FROM`.

```
1 SELECT * FROM chanson, genre_musical;
```

- La *jointure* recolle deux tables selon un critère logique (généralement l'égalité entre une clé étrangère et une clé primaire). Elle s'écrit formellement avec `JOIN ... ON ...`. Une alternative syntaxique consiste à réaliser un produit cartésien suivi d'une restriction `WHERE`.

```
1 SELECT *
2 FROM chanson
3 JOIN genre_musical ON chanson.id_genre = genre_musical.id;
```

- L'*autojointure* permet de joindre une table avec elle-même. Il est alors rigoureusement indispensable d'utiliser des *alias* (mot-clé `AS`) pour distinguer les deux instances de la table.

```
1 SELECT a.titre, b.titre
2 FROM chanson AS a, chanson AS b
3 WHERE a.duree + b.duree <= 540;
```

Agrégation et calculs

- Le langage SQL intègre des fonctions de calcul s'appliquant sur un ensemble de valeurs : `COUNT()` (dénombrement), `SUM()` (somme), `AVG()` (moyenne), `MAX()` et `MIN()`.

```
1 SELECT COUNT(DISTINCT artiste) FROM chanson;
2 SELECT AVG(duree) FROM chanson;
3 SELECT nom, MAX(duree) FROM chanson;
```

- La clause **GROUP BY** partitionne la table en sous-groupes (agrégats) en fonction des valeurs d'un attribut. La fonction d'agrégation spécifiée dans le **SELECT** s'applique alors indépendamment sur chaque sous-groupe.
- Lors d'une agrégation, le **SELECT** ne peut contenir *que* l'attribut de partitionnement et des fonctions d'agrégation.
- La clause **HAVING** applique une condition de restriction *après* l'opération de regroupement et de calcul, contrairement au **WHERE** qui filtre les lignes *avant* l'agrégation.

```
1 -- Compte, pour chaque artiste, le nombre de chansons synthwave, et ne
   garde que ceux qui en ont au moins 2.
2 SELECT artiste, COUNT(*)
3 FROM chanson
4 WHERE genre = 'synthwave'
5 GROUP BY artiste
6 HAVING COUNT(*) >= 2;
```